

STM32 UART - Introduction

Ankit Srivastava

EE2028

STM32L4S5xx Datasheet

STM32L4+ User Manual RM0432

B-L4S5I-IOT01A User Manual UM2708



OVERVIEW

- Introduction to UART
- UART- Character Framing
- UART Receiver and Transmitter
- UART in STM32L4S5VI

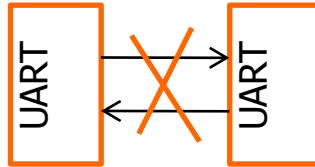
INTRODUCTION TO UART

UART is variously referred to as Serial, COM, tty, RS-232 (though RS-232 is in fact the physical layer) etc.

- Universal Asynchronous Receiver/Transmitter is a hardware for long-distance, asynchronous, serial, point-to-point, peer-to-peer, full-duplex communication
 - The above description is the typical use case. Different hardware may support a wide range of variations, such as synchronous transactions, simplex/half-duplex, multi-device bus like connections with addressing etc.
- UART Transmitter takes bytes of data and transmits the individual bits in a sequential fashion using a PISO (parallel-in, serial-out) shift register
- At the destination, UART Receiver re-assembles the bits into complete bytes using a SIPO (serial-in, parallel-out) register

INTRODUCTION TO UART

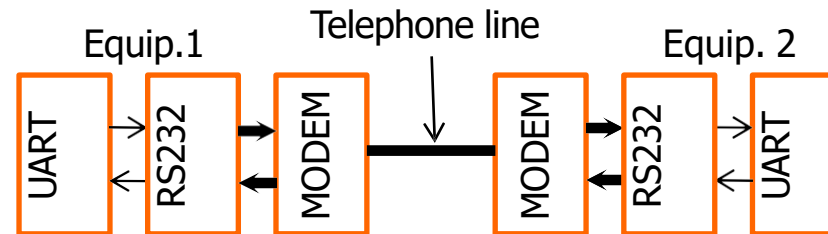
- UART refers to the hardware / logical layer and usually does not perform end-to-end communication between computers directly



- They are commonly used in conjunction with communication standards such as RS-232, ZigBee, Bluetooth etc., which includes some sort of physical layer
- Transmitting low-voltage digital signals directly over long distances is not practical

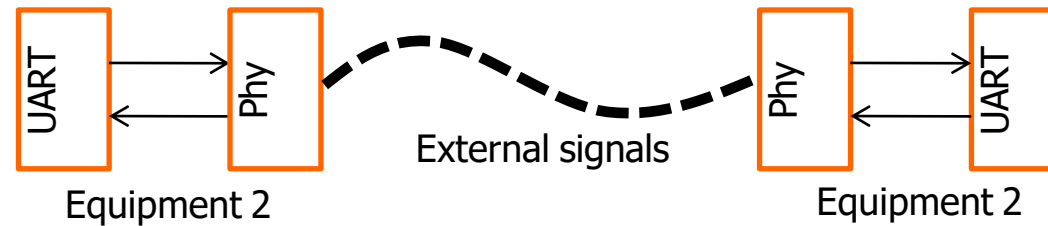
INTRODUCTION TO UART

- Transmitting low-voltage digital signals directly over long distances is not practical (why?)
- How are digital signals usually transmitted?
 - Many physical layer standards use modulation of a carrier signal (w/wo wires). Also called keying when the modulating signal is digital, as is the case with UART
 - modulation of audio signals with phone line modems
 - modulation of radio frequency (RF) signals
 - modulation of power line

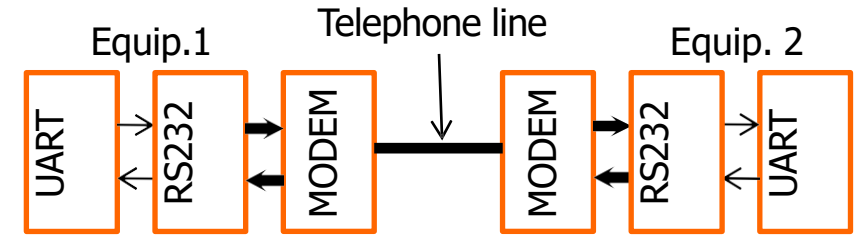


INTRODUCTION TO UART

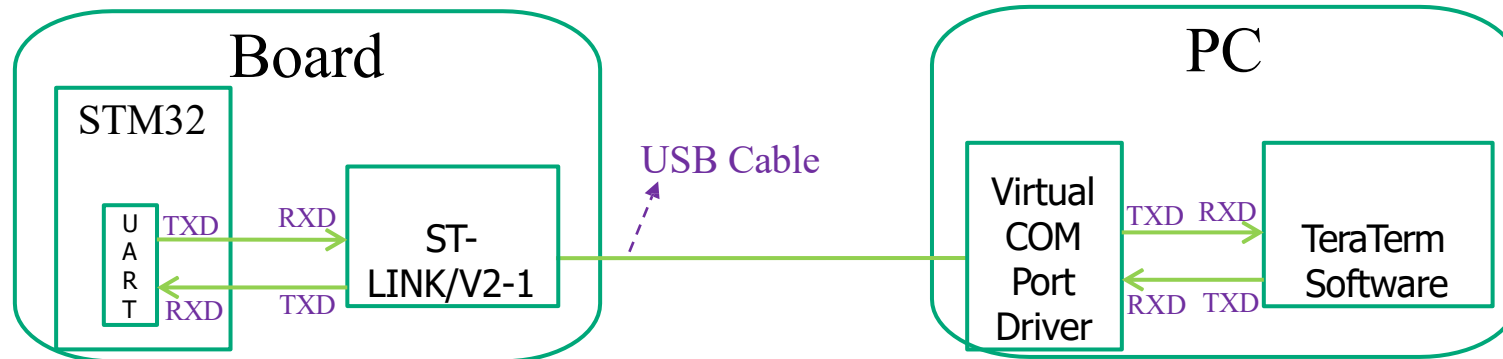
- External signals may be of many different forms:
 - Electrical signals in RS-232, RS-422 and RS-485
 - Some do not use electrical wires: optical fiber, IrDA (infrared), Bluetooth (wireless)



INTRODUCTION TO UART

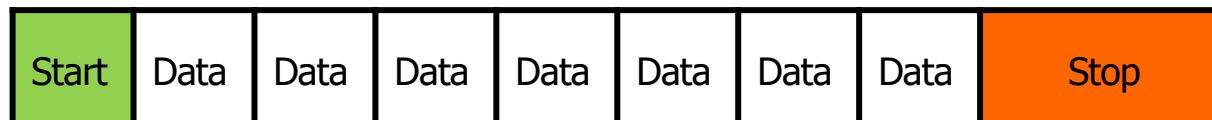


- The "old" dial-up internet access scenario using telephone modems (modulation of audio signals).
- These days, the modem is usually a cable/optical fibre/RF modem
- Many user software use UART for communication due to its simplicity
- It is translated to actual physical protocols such as USB by either hardware (USB-UART bridges such as ST-LINK/V2-1 on the board) or software (Virtual COM port driver on the PC)

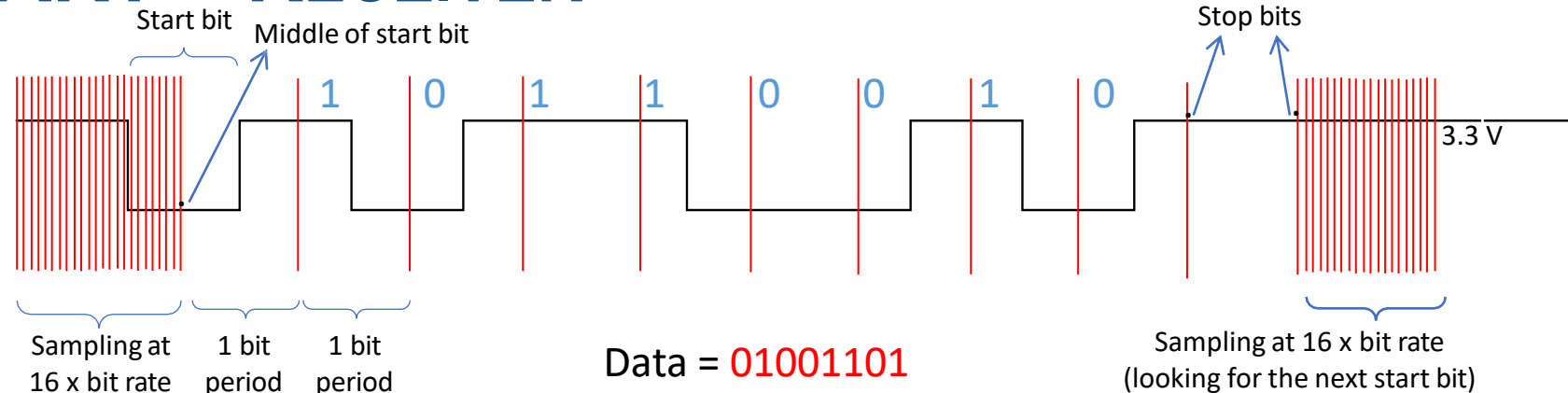


UART – CHARACTER FRAMING

- Each character is sent as a frame consisting of
 - a logic low start bit: signals the receiver that a new character is coming
 - a configurable number of data bits (called word length), usually 8. LSB is sent first (unlike I2C)
 - an optional parity bit (for detecting bit errors)
 - one or more logic high stop bits: signal the receiver that the character has completed
 - The line will remain high (default) until the next start bit
- Since the start bit is logic low (0) and the stop bit is logic high (1), there is always a clear demarcation between the previous character and the next one



UART – RECEIVER



The sampling scheme could vary depending on the specific UART implementation, but the goal is the same – to detect the incoming bit with high reliability in spite of having no shared clock with the transmitter to tell us when to sample

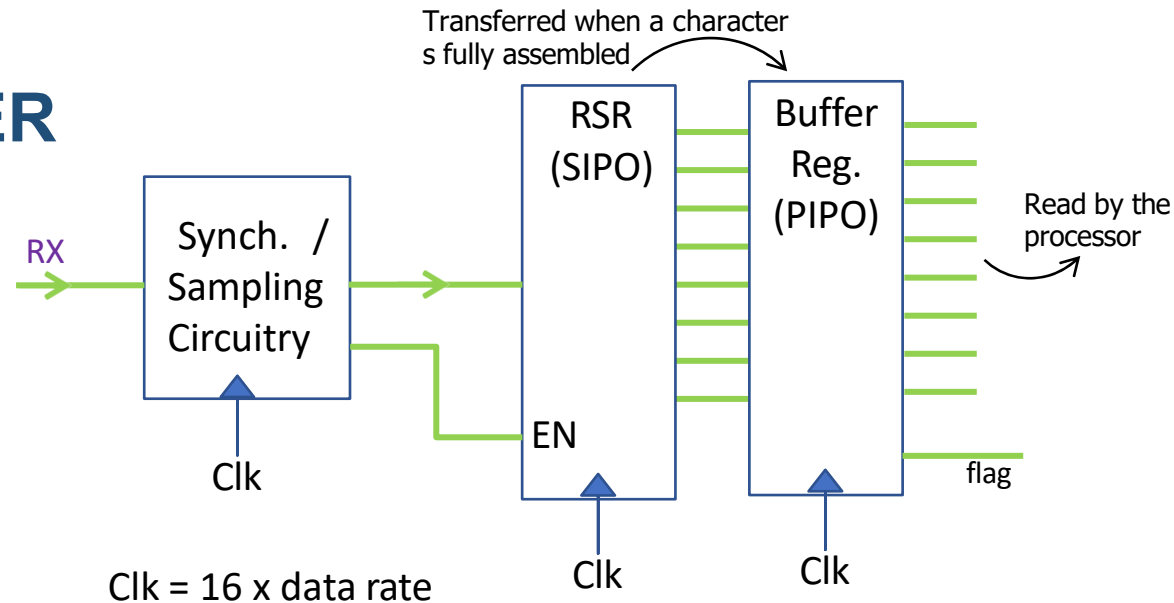
The terms 'character', 'byte', and 'word' are often interchangeably used in UART, though a byte is always 8 bits; a character/word is often, but not necessarily 8 bits. The parity bit may or may not be included in the word length depending on the context.

Middle of start bit = point at which 8 consecutive samples are zeros, assuming sampling clk = 16 x data rate
Typically, clk rate (of UART h/w) = at least 16x baud rate. Need not always be 16x; could be configurable

The two communicating systems should have identical data rates setting. This rate is usually known as Baud rate (strictly speaking, baud rate = symbol rate, which is not always equal to bit rate). The receiver knows how long a bit period is, but not exactly when it starts.

- All operations of UART hardware are controlled by a clock signal which runs at a multiple (say, 16) of the data rate - each data bit is as long as 16 clk pulses
- The receiver tests the state of the incoming signal on each clock pulse, looking for the beginning of the start bit
- If the apparent start bit lasts at least one-half of the bit time, it is valid and signals the start of a new character. If not, the spurious pulse is ignored
- After waiting a further bit time, the state of the line is again sampled and the resulting level clocked into a shift register

UART – RECEIVER



- After the required number of bit periods for the character length have elapsed, the contents of the shift register is a complete byte. Often, it is copied into a buffer register so that a new character can be received while waiting for the host processor to read this character from the buffer register
- UART will set a flag indicating new data is available, and may also generate a processor interrupt to request the host processor to read the received data
- In some common types of UART, a small first-in, first-out FIFO buffer memory is inserted between the receiver shift register and the host processor. This allows the host processor more time to read in the data (via interrupt or polling) and prevents loss of received data at high rates

UART – TRANSMITTER

- Transmission operation is simpler since it is under the control of the transmitting system
- As soon as data is deposited in the shift register after completion of the previous character, the UART hardware
 - generates a start bit
 - shifts the required number of data bits out to the line
 - generates and appends the parity bit (if used)
 - appends the stop bit(s)

UART – TRANSMITTER

- Since transmission of a single character may take a long time relative to CPU speeds, the UART will maintain a flag showing busy status so that the host system does not deposit a new character for transmission until the previous one has been completed; this may also be done with an interrupt
- Since full-duplex operation requires characters to be sent and received at the same time, practical UARTs use two different circuitry (incl. shift registers etc.) for transmitted characters and received characters

UART IN STM32L4S5VI

- STM32L4S5 has
 - three universal synchronous/asynchronous receiver transmitters (USART1, USART2 and USART3),
 - two universal asynchronous receiver transmitters (UART4, UART5),
 - a low-power UART (LPUART)
- The exact features depends on which of the above 3 is being used. USARTx is the most versatile, supporting almost all standard UART features.

UART IN STM32L4S5VI

- These standard features include
 - Configurable number of stop bits (0.5-2)
 - 7-9 bit data (word) length
 - Parity generation and checking (odd, even)
 - Autobaud rate detection (except LPUART)
 - 8 bytes TX/RX FIFO
- In addition, it also supports some features which are typically not supported by UARTs, such as
 - Multiprocessor communication with addressing
 - Synchronous mode etc.

UART REGISTERS AND HAL FUNCTIONS

- Control Registers : Configured by HAL_UART_Init()
 - Control register 1 (USART_CR1) : Parity settings, word length (inclusive of parity bit, if parity is used), transmit and receive enables, interrupt enables, fifo enable
 - Control register 2 (USART_CR2) : Number of stop bits, advanced features
 - Control register 3 (USART_CR3) : Sampling mode, error interrupt enable
 - Baud rate register (USART_BRR) : Baud rate setting
- Data Registers
 - Receive data register (USART_RDR) : CPU reads this to get the received data
 - Transmit data register (USART_TDR) : CPU writes data to be transmitted into it

UART REGISTERS AND HAL FUNCTIONS

- Status Register
 - Interrupt and Status Register (USART_ISR). Has status bits/flags such as
 - BUSY: Busy flag
 - TXE: Transmit data register empty
 - RXNE: Read data register not empty
 - Error flags like overrun error (new data received when previous data in USART_RDR is still unread by the CPU), parity error, framing error (stop bit not logic 1), etc.
- HAL_UART_Transmit() writes to USART_TDR if TXE bit in USART_ISR is set
- HAL_UART_Receive() reads from USART_RDR if RXNE bit in USART_ISR is set

CONCLUSION

- Introduction to UART
- UART- Character Framing
 - start bit, data bits, (optional) parity bit, (one or more) stop bits
- UART Receiver
- UART Transmitter
- UART in STM32L4S5VI
 - UART Registers
 - Control Register: USART_CR1, USART_CR2, USART_CR3, USART_BRR
 - Data Register: USART_RDR, USART_TDR
 - Status Register: USART_ISR
 - UART HAL functions: HAL_UART_Init, HAL_UART_Transmit, HAL_UART_Receive